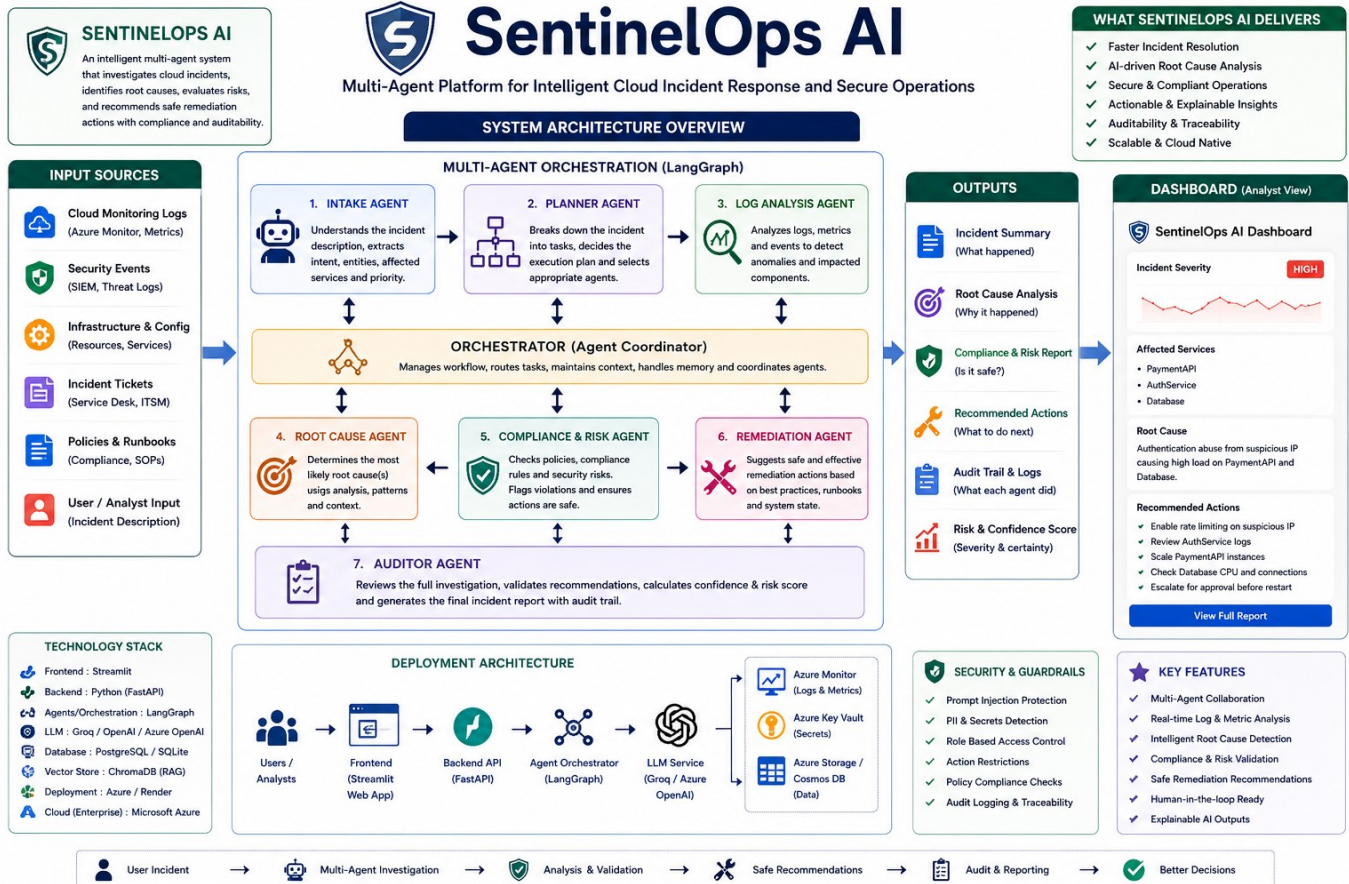


SentinelOps AI

Multi-Agent Platform for Intelligent Cloud Incident Response and Secure Operations

Project Build Guide | Junior FDE Pre-screening Assignment



Goal: Build a live Streamlit web app where specialized AI agents investigate a cloud incident, find likely root cause, enforce security/compliance guardrails, recommend safe remediation, and generate an audit-ready incident report.

1. What You Are Actually Building

You are building a deployed interactive web application, not a static website. The user enters a cloud incident description, and the system runs a coordinated multi-agent investigation. The final screen shows each agent output plus a final executive-style incident report.

Final Product

- A public Streamlit web app named SentinelOps AI.
- A GitHub repository with source code, README, data files, architecture diagram, and report.
- A 1-2 page written report summarizing architecture, security, implementation, and LLM collaboration.

- A live demo flow with at least one safe request and one blocked unsafe request.

User Demo Example

User input: "Payment API latency increased and failed logins spiked. Investigate possible root cause and recommend next steps."

Expected final output: severity = high, affected services = PaymentAPI/AuthService/Database, likely cause = suspicious authentication abuse plus backend pressure, safe actions = rate limiting, log review, scaling, approval before restart, audit status = approved with caution.

2. System Architecture

Component	Role in SentinelOps AI
User / Analyst Input	Incident description entered through the Streamlit UI.
Orchestrator	Coordinates all agents, manages workflow state, and passes outputs between agents.
Intake Agent	Classifies the incident, extracts services, severity hints, and intent.
Planner Agent	Breaks the investigation into tasks and decides which agents need to run.
Log Analysis Agent	Reads mock cloud logs/security events and identifies anomalies.
Root Cause Agent	Uses evidence from logs and patterns to infer likely causes.
Compliance & Risk Agent	Checks prompt injection, PII/secrets, blocked actions, and approval-required actions.
Remediation Agent	Recommends safe operational actions based on evidence and policy.
Auditor Agent	Validates the final response, adds risk/confidence scores, and generates audit trail.

Architecture Flow

User -> Streamlit UI -> Orchestrator -> Intake Agent -> Planner Agent -> Log Analysis Agent -> Root Cause Agent -> Compliance & Risk Agent -> Remediation Agent -> Auditor Agent -> Final Report

3. Free / Low-Cost Tools to Use

Layer	Tool	Why Use It	Cost Strategy
App UI	Streamlit	Fastest way to build a live dashboard without frontend complexity.	Streamlit Community Cloud is free for public apps.
Code Hosting	GitHub	Required for public repo submission and Streamlit deployment.	Free.
LLM API	Groq API	Fast hosted Llama models; useful for low-cost demos.	Free tier available, rate-limited.
Backup LLM	OpenAI API or Gemini	Use only if Groq fails or	Keep usage low;

		if you already have credits.	optional.
Cloud Story	Microsoft Azure	Enterprise-ready production architecture for Wipro/client-style explanation.	Use Azure for Students if available; \$100 credit for eligible students.
Vector/RAG	ChromaDB	Local vector search for policies/runbooks if you want RAG.	Free/local.
Data Storage	CSV + JSON + SQLite	Simple, reliable, and easy to explain.	Free.
Deployment	Streamlit Community Cloud	Simplest free public link for the live app.	Free.

Official Free Resource Links

- Streamlit Community Cloud: <https://streamlit.io/cloud>
- Streamlit docs: <https://docs.streamlit.io/>
- GroqCloud Console: <https://console.groq.com/>
- Groq rate limits: <https://console.groq.com/docs/rate-limits>
- Azure for Students: <https://azure.microsoft.com/en-us/free/students>
- Azure student offer details: <https://azure.microsoft.com/en-us/pricing/offers/ms-azr-0170p>
- LangGraph docs: <https://langchain-ai.github.io/langgraph/>
- ChromaDB docs: <https://docs.trychroma.com/>
- GitHub Docs: <https://docs.github.com/>

4. Data You Need

Use synthetic/mock enterprise data only. Do not use real customer data, personal data, secrets, or private logs.

File	Purpose	Sample Fields / Content
data/cloud_logs.csv	Simulates Azure Monitor metrics/logs.	timestamp, service, metric, value, status
data/security_events.csv	Simulates SIEM/security alerts.	timestamp, event_type, user, ip, severity
data/config_data.csv	Simulates infrastructure/service configuration.	service, replicas, region, db_connections, owner
data/compliance_rules.json	Defines blocked and approval-required actions.	blocked_actions, requires_approval
data/incident_policy.md	Explains incident handling policy/runbook.	severity definitions, escalation rules, safe actions
data/sample_incidents.csv	Demo incidents for quick testing.	incident_id, title, description, expected_severity

5. Exact Build Steps

Step 1 - Create GitHub repository: Create a public repo named sentinelops-ai. Add README.md, requirements.txt, app.py, agents/, data/, utils/, and docs/ folders.

Step 2 - Create mock data: Add cloud_logs.csv, security_events.csv, compliance_rules.json, and incident_policy.md. Keep all data synthetic.

Step 3 - Build Streamlit UI first: Create app.py with a title, incident text area, Run Investigation button, and output sections for each agent.

Step 4 - Build deterministic guardrails: Create utils/guardrails.py. Block prompt injection phrases, dangerous production actions, secrets/PII exposure, and approval-required actions.

Step 5 - Build simple agents: Create one Python file per agent. Each agent should accept the shared state dictionary and return structured outputs.

Step 6 - Connect the workflow: Start with sequential orchestration in app.py. Later, optionally convert to LangGraph if time allows.

Step 7 - Add LLM summaries: Use Groq for natural-language summaries only. Keep safety checks rule-based, not fully LLM-dependent.

Step 8 - Add audit log: Record each agent name, status, input summary, output summary, and timestamp.

Step 9 - Add demo test cases: Include one safe incident and one unsafe malicious request that gets blocked.

Step 10 - Deploy: Push to GitHub, connect repo to Streamlit Community Cloud, add API key as secret, and deploy app.py.

Step 11 - Prepare deliverables: Add architecture diagram, screenshots, README, 1-2 page report, and optional demo video link.

6. Suggested Repository Structure

```
sentinelops-ai/
├── app.py
├── requirements.txt
├── README.md
├── agents/
│   ├── intake_agent.py
│   ├── planner_agent.py
│   ├── log_analysis_agent.py
│   ├── root_cause_agent.py
│   ├── compliance_agent.py
│   ├── remediation_agent.py
│   └── auditor_agent.py
├── data/
│   ├── cloud_logs.csv
│   ├── security_events.csv
│   ├── config_data.csv
│   ├── compliance_rules.json
│   └── incident_policy.md
├── utils/
│   ├── guardrails.py
│   └── llm_client.py
└── docs/
    ├── architecture.png
    ├── report.md
    └── demo_script.md
```

7. What the App Screens Should Show

- Dashboard header: SentinelOps AI - Multi-Agent Cloud Incident Response.
- Incident input card with sample incident dropdown and text area.
- Agent workflow timeline showing Intake, Planner, Log Analysis, Root Cause, Compliance, Remediation, Auditor.
- Live agent output cards with status: completed, warning, blocked, or needs approval.
- Final incident report with severity, affected services, root cause, safe actions, risk score, confidence score, and audit trail.
- Guardrail panel showing whether any request was blocked or required human approval.

8. Guardrails You Must Explain

- Input validation: reject empty, overly long, or malicious requests.
- Prompt injection protection: block phrases like "ignore previous instructions" or "bypass policy".
- PII/secrets protection: detect SSN/API key/password patterns and mask them.
- Action restrictions: block destructive actions like deleting databases or disabling logging.
- Human approval: production restart, firewall changes, and scaling down databases require approval.
- Audit logging: every agent action is logged for traceability.
- Least privilege: each agent only has access to the tools/data it needs.

9. Implementation Strategy

Recommended MVP approach: build the entire workflow with Python functions first, then optionally add LangGraph if time allows. This keeps the project finishable while still demonstrating multi-agent architecture clearly.

Priority	Feature	Build Decision
Must Have	Working Streamlit app	Required for live demo.
Must Have	5-7 specialized agents	Shows actual multi-agent collaboration.
Must Have	Guardrails and blocked unsafe request	Directly addresses assignment security requirements.
Must Have	Audit trail	Shows enterprise readiness and explainability.
Should Have	LLM-generated final report	Improves polish and realism.
Optional	LangGraph orchestration	Good if time allows; not required for MVP.
Optional	ChromaDB RAG over policy docs	Useful if you want stronger retrieval story.

10. Realistic Timeline

Day	Goal
Day 1	Create repo, data files, Streamlit UI, and first three agents.
Day 2	Add remaining agents, guardrails, audit log, and sample test cases.

Day 3	Add LLM summaries, improve UI, add screenshots, and deploy.
Day 4	Write README, report, architecture explanation, and demo script.
Day 5	Practice presentation, record backup demo video, and submit GitHub link.

11. Final Deliverables Checklist

- Public GitHub repository link.
- Live Streamlit app link.
- 1-2 page written report.
- Architecture diagram with SentinelOps AI name.
- README with setup, deployment, architecture, guardrails, and demo examples.
- At least two demo scenarios: one safe investigation and one blocked unsafe request.
- Optional recorded demo video as backup.

12. Short Presentation Script

Opening: SentinelOps AI is a multi-agent cloud incident response platform that helps enterprise operations teams investigate incidents, identify root causes, enforce compliance guardrails, and recommend safe remediation actions.

Architecture: The system uses an orchestrator that routes the request through specialized agents: Intake, Planner, Log Analysis, Root Cause, Compliance, Remediation, and Auditor. Each agent has a clear responsibility and boundary.

Security: Safety is built into the workflow through deterministic guardrails, prompt-injection checks, PII/secrets detection, blocked actions, approval-required actions, and audit logging.

Implementation: The prototype is built with Python and Streamlit, uses synthetic cloud/security logs, and can optionally use Groq/OpenAI for natural language summaries. The production version would run on Azure with Azure Monitor, Azure Key Vault, Azure OpenAI, Entra ID, and managed storage.

13. Source Notes

The free/low-cost recommendations are based on official platform pages available as of May 18, 2026: Streamlit Community Cloud states apps can be deployed and shared for free; Azure for Students lists \$100 credit and no credit card for eligible students; Render lists a free web service tier; Groq describes free-start/API access with rate limits. Always verify account-specific limits before deployment.